Gujarat Vidyapith		
Department of Computer Science		
Course : Msc(IT) Semester-I	**Year : 2026-27**	**Subject Teacher : Prof. (Dr.) Ajay Parikh**
Sub : Lab based on Python Programming and Application Development		**Enrollment No.:_____**

Practical No.	Lab Activity	Date Assigned	Date Submission	Sign	Grade
1	**Consecutive Duplicate Detector** Accept **N** integers. Display only those numbers that appear **consecutively** more than once. Input: 1 2 2 3 4 4 4 5 Output: 2 4	23/7/2026	6/8/2026		
2	**Missing Roll Number** Roll numbers should be from **1 to N**. One roll number is missing. Find the missing roll number without sorting. Example: 1 2 3 5 6 Output: 4	23/7/2026	6/8/2026		
3	**Password Strength Analyzer** Accept password. Check ● uppercase ● lowercase ● digit ● special character ● repeated consecutive characters Display all failed rules.	30/7/2026	6/8/2026		
4	Paragraph Analyser using python The program should: ● Split the paragraph into words and store them in a list. ● Display: ○ Total number of words ○ Number of unique words ○ Longest word ○ Shortest word ○ Words appearing more than once	6/8/2026	7/8/2026		

5	**Student Rank Processing Engine** Develop a Python application that reads student records from User input. Each record contains: ● Roll No ● Name ● Marks in 5 subjects The program must: ● Compute total and percentage. ● Assign grades. ● Handle ties while assigning ranks (students with equal marks receive the same rank). ● Display the next rank correctly (e.g., 1,2,2,4).	7/8/2026	13/8/2026		
6	**Student Rank Processing Engine using Functions and Modules** Develop a Python application to process and rank student records stored in an Excel file. Organize the application using user-defined modules and functions. Input The input Excel file contains the following fields: ● Roll No ● Name ● Marks in Subject 1 ● Marks in Subject 2 ● Marks in Subject 3 ● Marks in Subject 4 ● Marks in Subject 5 Create the following modules: student.py Implement functions to: ● Read student records from an Excel file. ● Calculate the total marks and percentage for each student. ● Assign grades based on the percentage. ● Return the processed student records. ranking.py Implement functions to: ● Rank students based on total marks. ● Handle tied ranks correctly (e.g., 1, 2, 2, 4). ● Sort students in descending order of marks. ● Return the ranked student records. report.py Implement functions to: ● Display the ranked student records in a formatted table. ● Export the ranked results to a new Excel file. main.py The main program should: 1. Import the required functions from all modules.	13/8/2026	21/8/2026		

<table>
<tr><td></td><td>2. Read student records from the input Excel file.
3. Calculate total marks, percentage, and grade for each student.
4. Generate student ranks while correctly handling ties.
5. Display the ranked result in rank order.
6. Export the final ranked result to another Excel file.
7. Handle file-related errors and invalid data using exception handling.</td><td></td><td></td><td></td><td></td></tr>
<tr><td>7</td><td>Course Registration Conflict Detector using Object-Oriented Programming
Develop a Python application to help a college identify timetable conflicts when students register for multiple courses.
Create the following classes:
● Course
○ Course Code
○ Day of the Week
○ Start Time
○ End Time
● Student
○ Student ID
○ Student Name
○ List of Registered Courses
Implement appropriate constructors (__init__()) to initialize all objects.
The application should:
1. Accept details of N students.
2. For each student, accept details of multiple registered courses.
3. Detect whether any registered courses overlap on the same day.
4. Display all conflicting course pairs.
5. If no conflict exists, display the student's timetable.
6. Calculate and display the total number of registered courses for each student.
7. Override the __str__() method to display student and course information in a readable format.
8. Handle invalid time entries or incorrect input using exception handling.</td><td>03/09/2026</td><td>15/10/2026</td><td></td><td></td></tr>
<tr><td>8</td><td>Students and working professionals often need to manage daily tasks such as assignments, meetings, project work, and personal activities. You need to build a simple To-Do List web app to add, track, update, and delete tasks using Flask.
Data Schema / Model<table><tr><th>Field</th><th>Type</th><th>Description</th><th>Example</th></tr><tr><td>id</td><td>Integer</td><td>Primary key</td><td>1</td></tr></table></td><td>24/09/2026</td><td>15/10/2026</td><td></td><td></td></tr>
</table>

title	String (150)	Task title	"Complete Python Assignment"
description	String (300)	Details about the task	"Complete Flask practical"
priority	String (20)	Task priority	"High", "Medium", "Low"
is_completed	Boolean	Task completion status	True / False
due_date	Date	Task deadline	2026-09-25

🛠️ **Required Routes & Functionality**

- GET /: Display two sections: **Pending Tasks** and **Completed Tasks**.
- POST /add: Form to add a new task with title, description, priority, and due date.
- GET /toggle/<id>: One-click button to toggle a task between **Pending** and **Completed**.
- POST /update/<id>: Update the details of an existing task.
- POST /delete/<id>: Remove a task from the to-do list.

9	Develop a Personal Task Tracker web application using **FastAPI** that allows users to log in, manage their daily tasks, and track task completion. The application must use an SQLite database to permanently store all task data, sessions to maintain the logged-in user's information, and cookies to remember the user's preferences. Each user should be able to view and manage their own tasks. Task data must remain available even after the application is restarted. **Data Schema / Model** Task Table — SQLite Database				08/10/2026	22/10/2026	

Field	Type	Description	Example
id	Integer	Primary key	1
title	String (150)	Task title	"Complete Python Assignment"
description	String (300)	Task details	"Complete FastAPI practical"

priority	String (20)	Task priority	"High", "Medium", "Low"
is_completed	Boolean	Task completion status	False
created_by	String (100)	Name of the task owner	"Hasmukh"
created_at	DateTime	Task creation date and time	Current date and time

Session and Cookie Data

- Session: Store the logged-in user's name or user ID.
- Cookie: Store the user's preferred theme, such as light or dark.
- SQLite: Store all tasks permanently in a database file named tasks.db.

Required Routes & Functionality

- GET /: Display the logged-in user's tasks, separated into Pending Tasks and Completed Tasks.
- GET /login: Display a simple login form to enter the user's name.
- POST /login: Store the user's information in the session.
- POST /add: Add a new task and save it in the SQLite database.
- GET /toggle/{id}: Toggle a task between Pending and Completed, updating the database.
- GET /edit/{id}: Display a form to edit an existing task.
- POST /update/{id}: Update the task details in SQLite.
- POST /delete/{id}: Delete a task from SQLite.
- GET /set-theme/{theme}: Store the selected theme in a cookie.
- GET /logout: Clear the session and log out the user.

Database Requirements

1. Use SQLite with a database file named tasks.db.
2. Create the tasks table automatically when the application starts.
3. Use SQLite queries or SQLAlchemy ORM to perform database operations.

	4. Insert new tasks into the database. 5. Retrieve tasks from the database when the home page loads. 6. Update and delete records in the database when requested. 7. Ensure that tasks remain stored after the application is restarted. 8. Display only the tasks belonging to the currently logged-in user.				

Grade	Marks
A	18 - 20
B	14 - 17
C	11 - 13
D	8 - 10
E	5 - 7
F	1 - 4
S	Absent